SLIIT
Discover Your Future

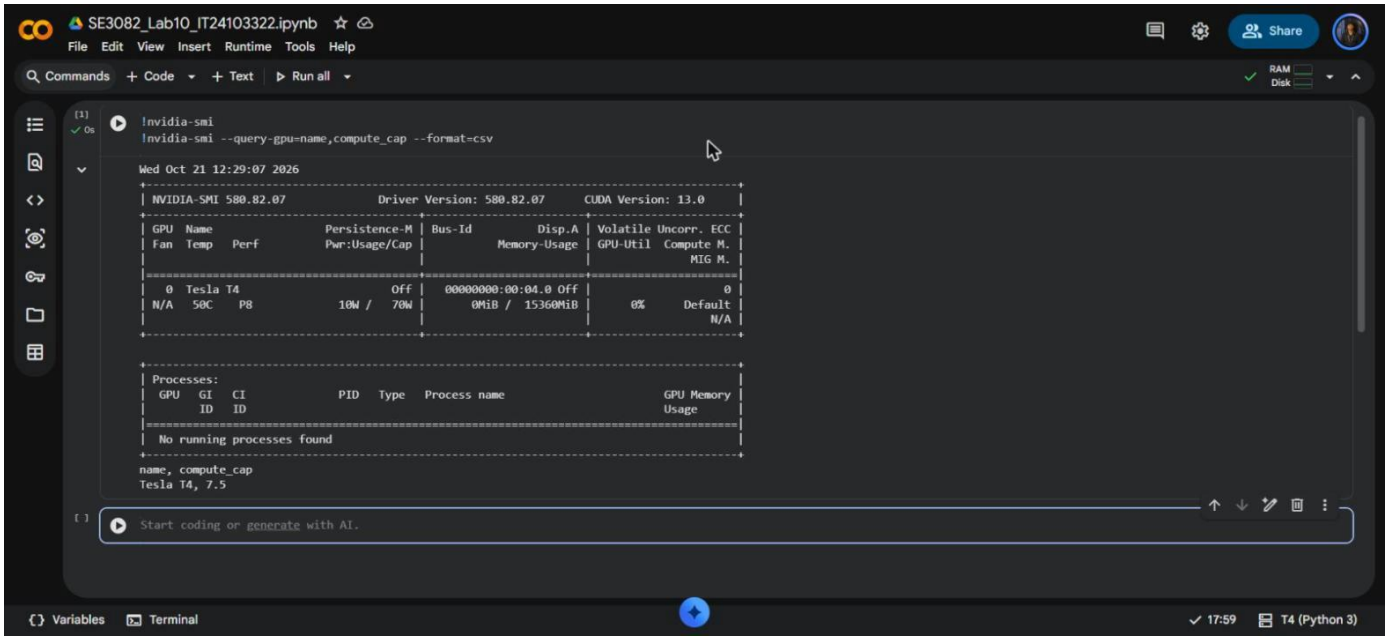
# BSc (Hons) in Computer Science
# Year 3

**Lab Sheets**

**SE3082 – Parallel Computing** **Semster 1, 2026**

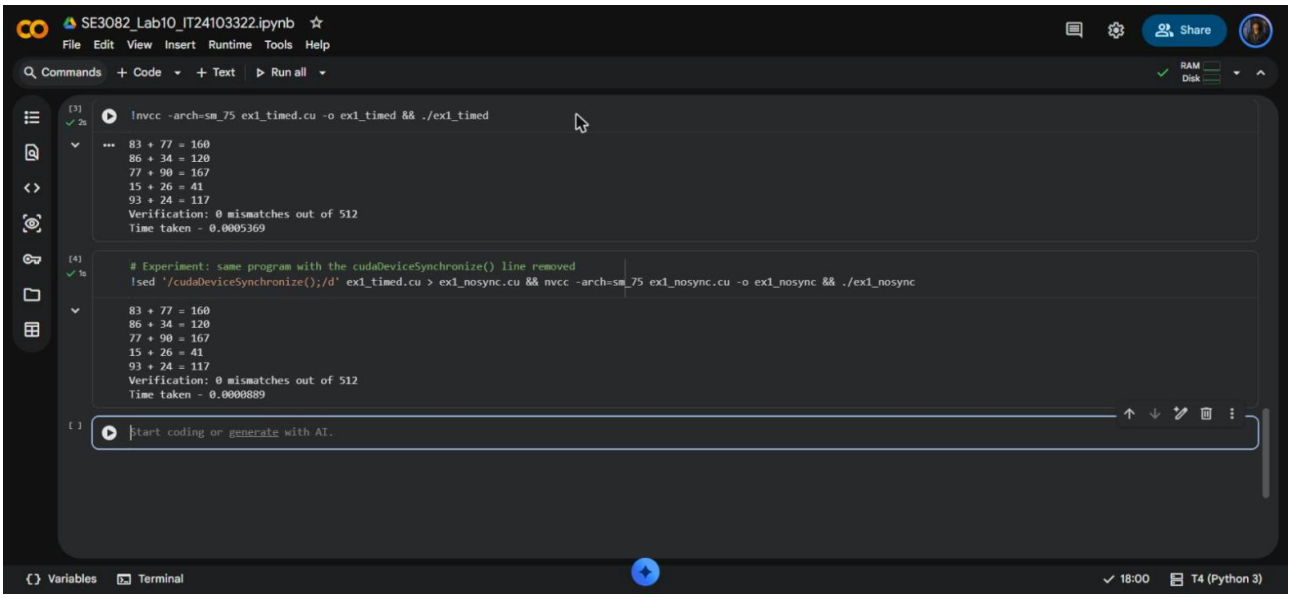
**IT24103322**

**Exercise 1**




A new notebook SE3082_Lab10_IT24103322 was created in Colab with the T4 GPU runtime. nvidia-smi shows a Tesla T4 with compute capability 7.5, so every program was compiled with -arch=sm_75.

a. **Compile and run the timed program.**

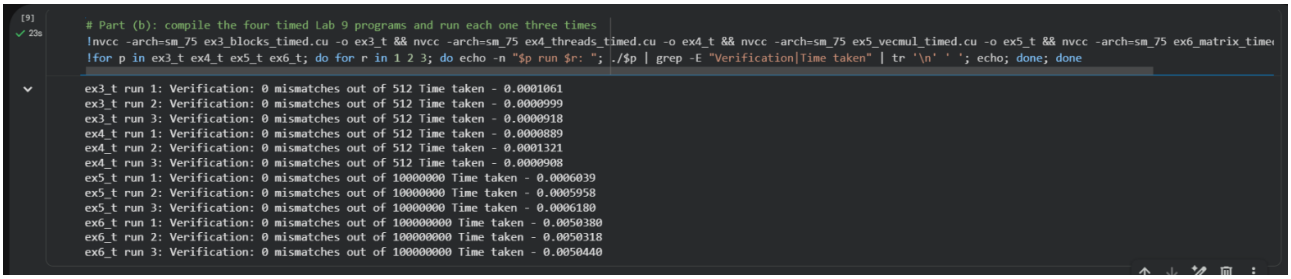



The timer uses cpuSecond() (gettimeofday) around the kernel launch, with cudaDeviceSynchronize() before the clock is stopped. The first five results are 83 + 77 = 160 … 93 + 24 = 117, Verification: 0 mismatches out of 512, and Time taken - 0.0005369 s.

**Experiment – remove cudaDeviceSynchronize(). What happens to the time? Does the answer still verify?**

The second cell copies the program without the cudaDeviceSynchronize() line. The time dropped from 0.0005369 s to 0.0000889 s. A kernel launch is asynchronous: the CPU starts the GPU and carries on straight away, so without the sync only the launch is timed, not the work. The answer still verifies (0 mismatches) because the cudaMemcpy back to the host waits for the kernel to finish before it copies.

b. **Add the timer to the Lab 9 programs (Exercises 3, 4, 5 and 6) and record their times.**




| Lab 9 program | Launch configuration | Run 1 (s) | Run 2 (s) | Run 3 (s) |
|---|---|---|---|---|
| Exercise 3 | 512 blocks × 1 thread | 0.0001061 | 0.0000999 | 0.0000918 |
| Exercise 4 | 1 block × 512 threads | 0.0000889 | 0.0001321 | 0.0000908 |
| Exercise 5 | 19,532 blocks × 512 threads | 0.0006039 | 0.0005958 | 0.0006180 |
| Exercise 6 | 313 × 625 blocks of 32 × 16 | 0.0050380 | 0.0050318 | 0.0050440 |

Every run reported 0 mismatches.

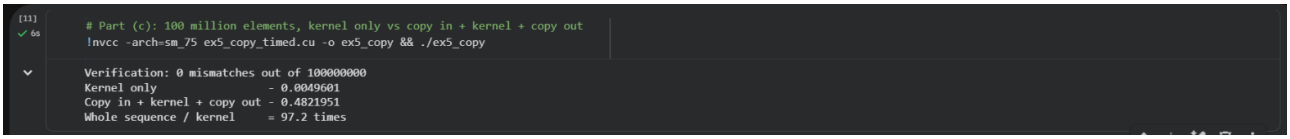
1. **512 blocks of one thread or one block of 512 threads – which is faster, and why?**

   One block of 512 threads should be faster, because its threads fill 16 complete warps, while 512 one-thread blocks use 512 warps with 31 of 32 lanes idle. In the measurements both took about 0.1 ms and the order changed between runs: with only 512 additions the launch overhead is larger than the work, so the difference is hidden in the noise.

2. **Exercise 6 has 10 times as many elements as Exercise 5. Does it take 10 times as long?**

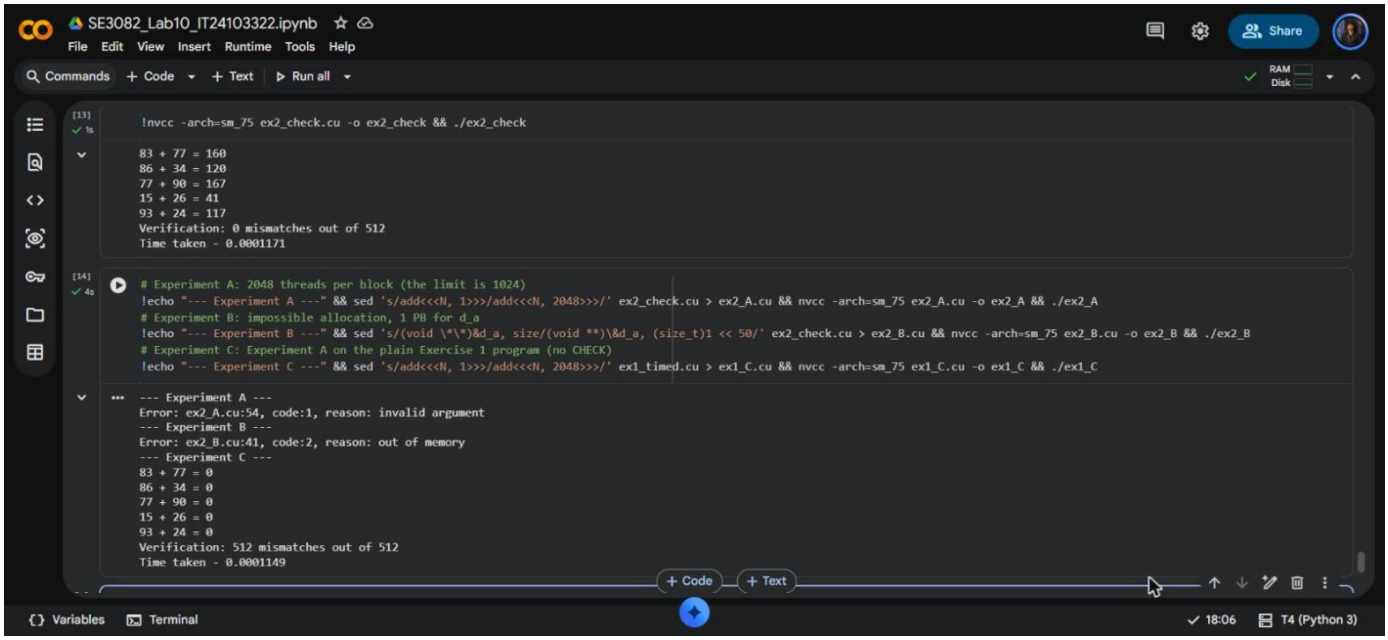
   Close to it: about 5.04 ms against 0.60 ms, which is 8.4 times. Both kernels read two numbers and write one per element, so the time grows with the amount of data. It is a little under 10 times because part of the Exercise 5 time is the fixed cost of the launch.

c. **Kernel time versus copy time for 100 million elements.**




   With N = 100,000,000 (about 400 MB per array) the kernel alone took 0.0049601 s, while copy in + kernel + copy out took 0.4821951 s, about 97 times longer. The 1.2 GB of data has to travel over the PCIe bus, which is much slower than the GPU's own memory. So the GPU is only worth using for work that does a lot of calculation per number copied, or when the data can stay on the GPU for many kernels; a single multiply per element is mostly copy time.

**Exercise 2**



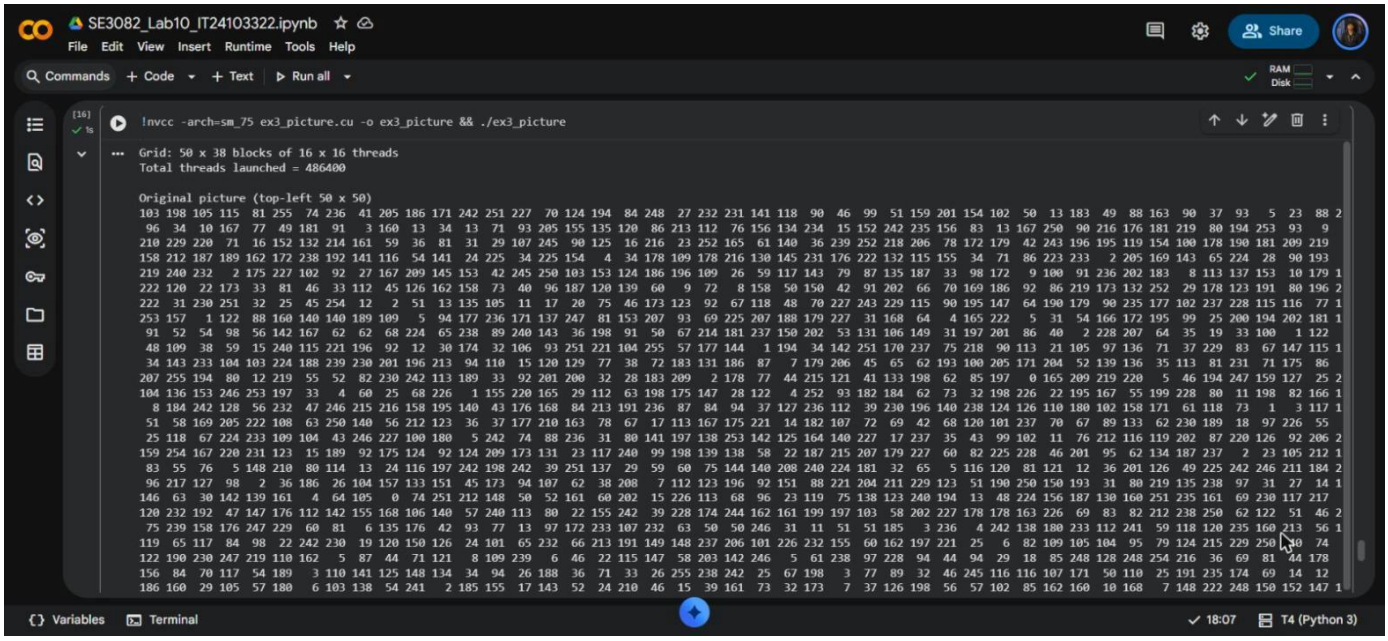
Every cudaMalloc, cudaMemcpy and cudaFree call was wrapped in CHECK(...), and CHECK(cudaGetLastError()) and CHECK(cudaDeviceSynchronize()) were added after the launch. The output is the same as Exercise 1 (0 mismatches out of 512, Time taken - 0.0001171).

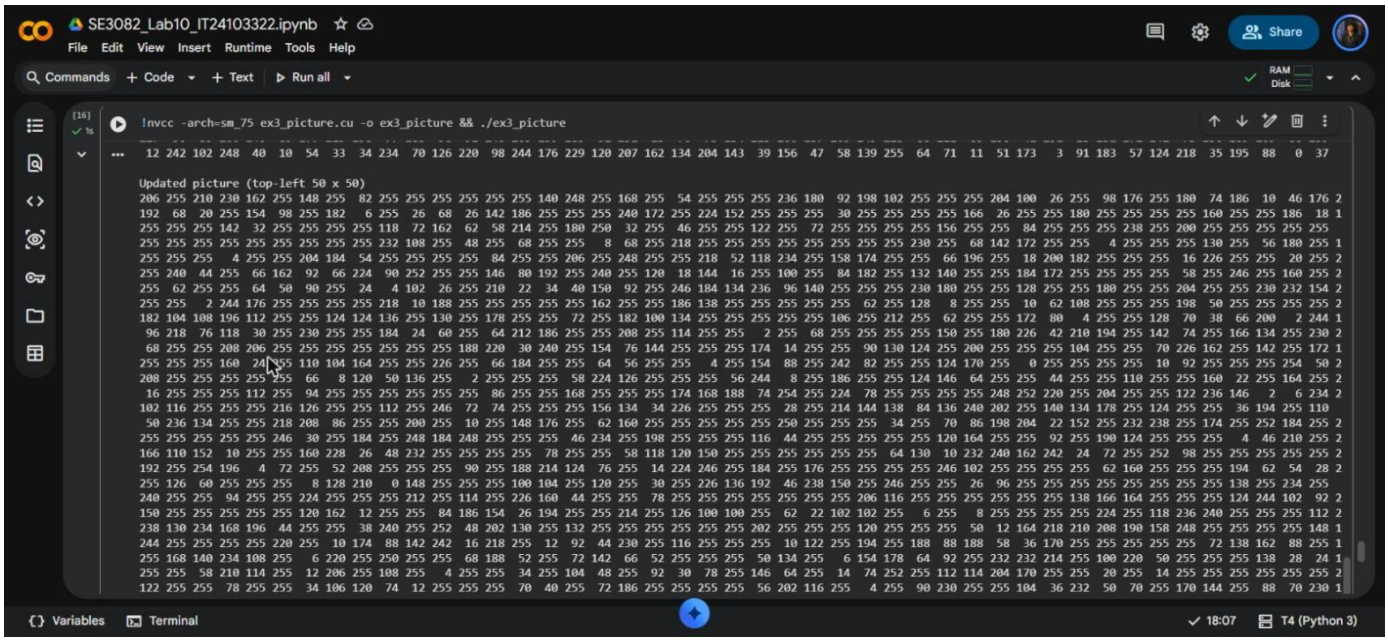
| Experiment | Change | Output |
|---|---|---|
| A – too many threads | add<<<N, 2048>>> | Error: ex2_A.cu:54, code:1, reason: invalid argument |
| B – impossible allocation | cudaMalloc of (size_t)1 << 50 | Error: ex2_B.cu:41, code:2, reason: out of memory |
| C – no CHECK | Experiment A on Exercise 1 | No error; every c is 0, 512 mismatches out of 512 |

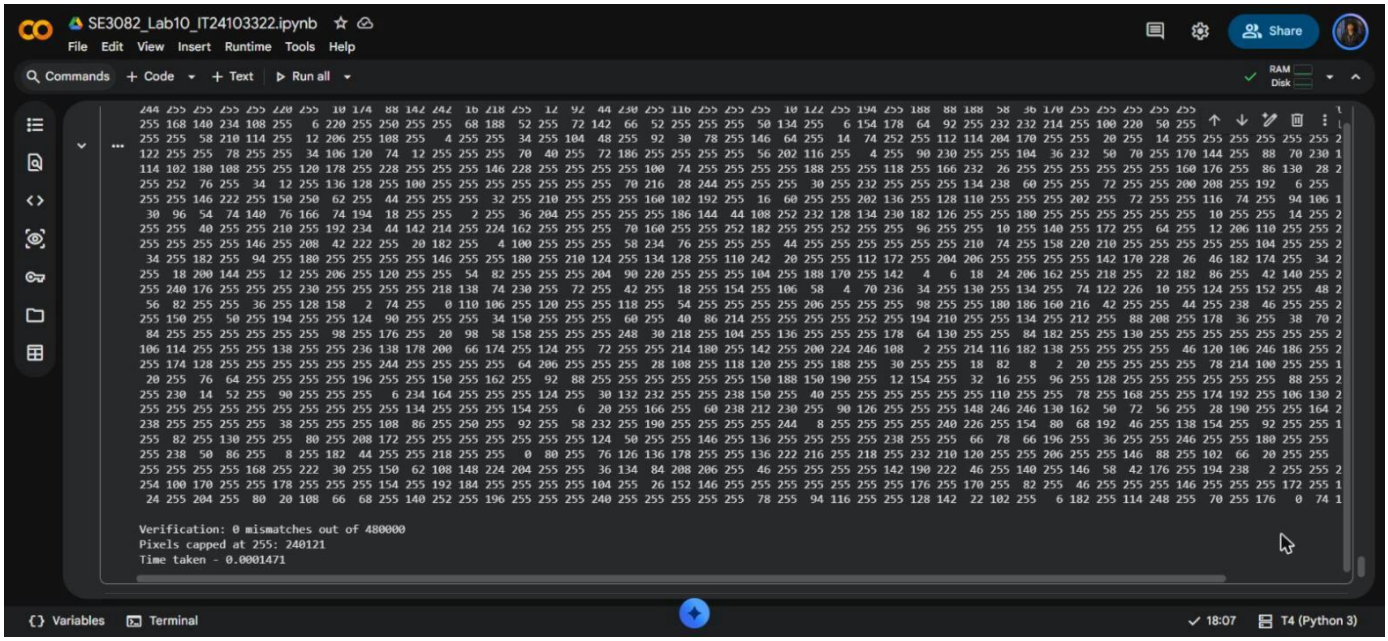
Experiment A was caught by CHECK(cudaGetLastError()) on line 54 (the launch row of the table), because 2048 threads per block is over the limit of 1024. On this CUDA 13.0 runtime the error is reported as code 1, invalid argument, instead of the code 9 shown in the sheet. Experiment B was caught on line 41, the first cudaMalloc (the row for calls that return a value).

In Experiment C the program printed no error at all; it carried on, printed 83 + 77 = 0 and so on, and only the verification loop showed that something was wrong. Without that loop the failure would not be noticed. The do { ... } while (0) around the macro makes CHECK(x); behave like one normal statement, even inside an if without braces.

**Exercise 3**





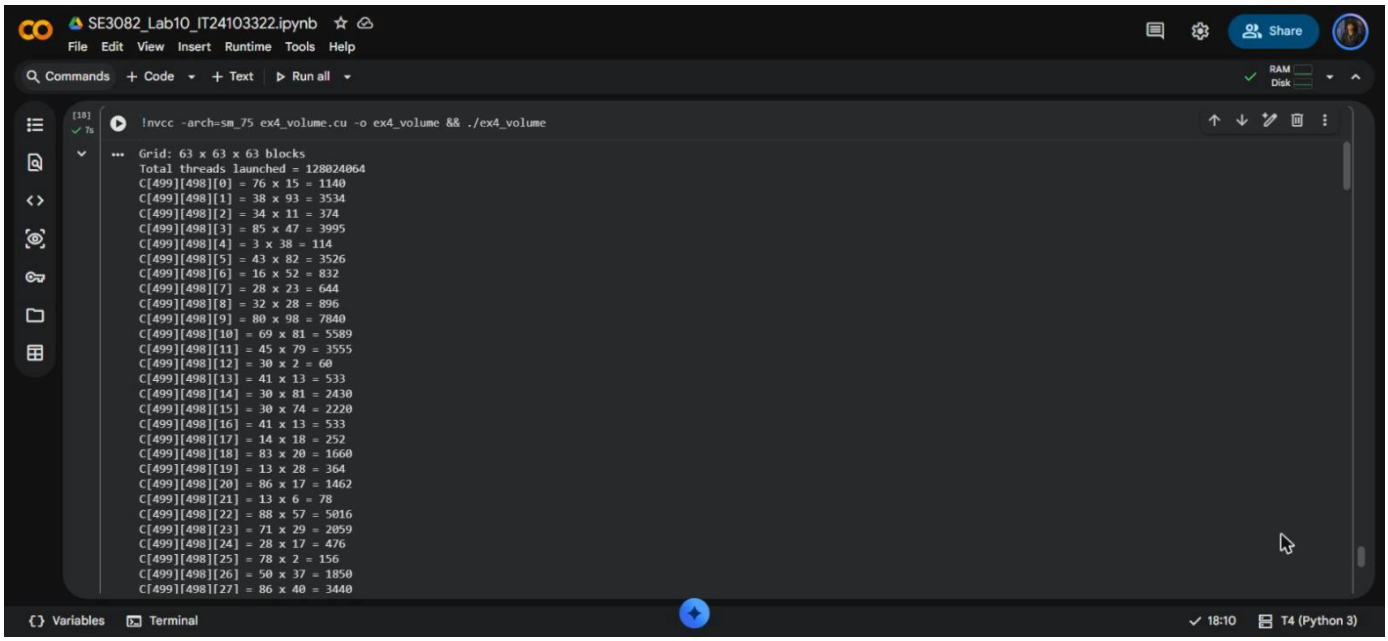

The 800 × 600 picture is one flat array of 480,000 unsigned char values, filled with rand() % 256. dim3 threadsPerBlock(16, 16) gives 256 threads per block, and the grid is rounded up in both directions: 800 / 16 = 50 blocks across and 600 / 16 = 37.5 → 38 blocks down. The PictureKernel uses Row from y and Col from x, works on pixel Row * width + Col only if both are inside the picture, and doubles the value with a cap at 255. The top-left 50 × 50 pixels were printed before and after.

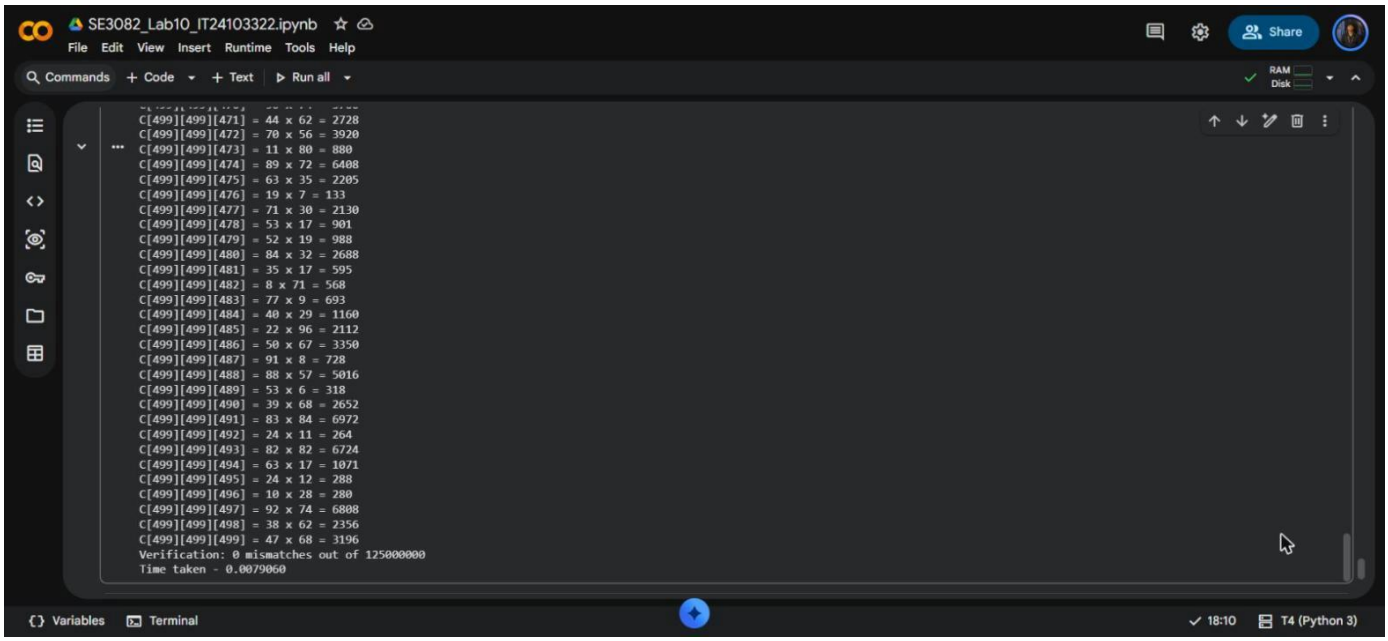
| Item | Expected value | My result |
|---|---|---|
| Grid | 50 x 38 blocks of 16 x 16 threads | 50 x 38 blocks of 16 x 16 threads |
| Total threads launched | 486,400 | 486400 |
| First ten original values | 103 198 105 115 81 255 74 236 41 205 | 103 198 105 115 81 255 74 236 41 205 |
| First ten updated values | 206 255 210 230 162 255 148 255 82 255 | 206 255 210 230 162 255 148 255 82 255 |
| Verification | 0 mismatches out of 480000 | 0 mismatches out of 480000 |
| Pixels capped at 255 | about half (240,121) | 240121 |

1. **Roughly how many of the 480,000 pixels will be capped at 255?**

   About half. Every original value from 128 to 255 becomes 256 or more when doubled, and that is half of the 256 possible values. 240,121 pixels (50.0 %) were capped. Without the cap 2 × 200 = 400 would wrap around to 144 in an unsigned char.

**Exercise 4**






The volume is 500 × 500 × 500 = 125,000,000 ints (500 MB per matrix), stored as flat arrays. Each block is dim3(8, 8, 8) = 512 threads, and the grid is rounded up in all three directions: 500 / 8 = 62.5 → 63. In the kernel x, y and z each come from blockIdx * blockDim + threadIdx, the element is computed only if x < nx, y < ny and z < nz, and the flat index is (z * ny + y) * nx + x. The last 1000 elements were printed in memory order.

| Item | Expected value | My result |
|---|---|---|
| Grid | 63 x 63 x 63 blocks | 63 x 63 x 63 blocks |
| Total threads launched | 128,024,064 | 128024064 |
| First of the last 1000 | C[499][498][0] = 76 x 15 = 1140 | C[499][498][0] = 76 x 15 = 1140 |
| Last result | C[499][499][499] = 47 x 68 = 3196 | C[499][499][499] = 47 x 68 = 3196 |
| Verification | 0 mismatches out of 125000000 | 0 mismatches out of 125000000 |

1. **Compare the time with Lab 9 Exercise 6. Is the 3D kernel faster, slower or about the same?**

   The 3D kernel took 0.0079060 s for 125 million elements; the 2D kernel took about 0.00504 s for 100 million. Per element the 3D kernel is about 25 % slower (63 ps against 50 ps). The work per element is the same, so the difference most likely comes from the thread layout: with 8 threads in x, one warp of 32 threads covers 4 short rows of 8 numbers instead of 32 numbers next to each other, so its memory accesses are less coalesced, and each thread also does more index arithmetic.

The notebook SE3082_Lab10_IT24103322.ipynb in my Google Drive contains the working cells for every exercise with the outputs visible.